Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información

Programación II

Catedrático: JOSSEPH EMMANUEL TURNIL MURGA

Proyecto final

Sistema de Gestión de Órdenes de Trabajo

Taller Mecánico "El Último Arranque"

Estudiante

Osmar Josué Menéndez Morales

Carné: 7690-15-15347

Guatemala, 22 de octubre de 2026

Índice

1. Introducción 4
2. Definición del problema 5
3. Objetivos 6
3.1 Objetivo general 6
3.2 Objetivos específicos 6
4. Tecnologías utilizadas 7
5. Base de datos 8
5.1 Descripción del modelo 8
5.2 Diagrama entidad-relación 8
5.3 Diccionario de datos 8
CLIENTE 8
MECANICO 9
VEHICULO_TALLER 9
REPUESTO 9
ABASTECIMIENTO 10
SERVICIO 10
ORDEN_TRABAJO 10
ORDEN_SERVICIO 11
ORDEN_REPUESTO 11
5.4 Prueba de conexión 11
5.5 Script SQL de creación y datos de prueba 11
6. Diagrama de clases UML 15
6.1 Clases del modelo 15
6.2 Arquitectura del sistema 15
7. Aplicación de la programación orientada a objetos 17
8. Módulos del sistema 18
8.1 Clientes 18
8.2 Vehículos 19
8.3 Mecánicos 19
8.4 Repuestos 20
8.5 Servicios 21
8.6 Órdenes de trabajo 21
8.7 Transacciones 22

9. Casos de uso 24
10. Reportes 25
10.1 Importación y exportación CSV 26
11. Repositorio Git 27
11.1 Historial de commits 27
12. Conclusiones 28

1. Introducción

El presente documento describe el desarrollo de una aplicación de escritorio para administrar las operaciones del taller mecánico "El Último Arranque". La aplicación fue desarrollada en Java con interfaz gráfica Swing y utiliza Oracle Database como sistema de almacenamiento, con conexión mediante JDBC.

El sistema permite gestionar clientes, vehículos, mecánicos, repuestos y servicios, y su módulo principal administra las órdenes de trabajo desde la recepción del vehículo hasta la generación del comprobante de pago.

2. Definición del problema

El taller recibe diariamente vehículos para mantenimiento y reparación. Necesita llevar el control de sus clientes y vehículos, asignar mecánicos a cada reparación, controlar el inventario de repuestos y conocer el estado de cada trabajo.

Cuando un vehículo ingresa se genera una orden de trabajo con los datos del vehículo, el mecánico asignado y la descripción del problema. Durante la reparación se registran los servicios realizados y los repuestos utilizados, verificando que existan unidades suficientes en inventario. Al finalizar, el sistema calcula el costo total (mano de obra más repuestos) y genera un comprobante para el cliente.

3. Objetivos

3.1 Objetivo general

Desarrollar una aplicación de escritorio en Java que administre las operaciones del taller mecánico mediante una interfaz gráfica y una base de datos Oracle.

3.2 Objetivos específicos

- Registrar, modificar, consultar y eliminar clientes, vehículos, mecánicos, repuestos y servicios.
- Administrar el ciclo de vida de las órdenes de trabajo: PENDIENTE, EN PROCESO, FINALIZADA y CANCELADA.
- Controlar el inventario de repuestos con operaciones transaccionales que garanticen la consistencia de los datos.
- Calcular automáticamente los costos de reparación y generar el comprobante final.
- Generar reportes en JTable con opción de exportación a HTML y CSV.
- Aplicar los principios de la programación orientada a objetos.

4. Tecnologías utilizadas

Herramienta	Uso en el proyecto
Java (JDK)	Lenguaje de programación de la aplicación
Swing	Interfaz gráfica de usuario
Oracle Database	Almacenamiento de la información
JDBC (ojdbc)	Conexión entre Java y Oracle, con PreparedStatement
Apache NetBeans	Entorno de desarrollo
SQL Developer	Creación y administración de la base de datos
Git y GitHub	Control de versiones y trabajo colaborativo

5. Base de datos

5.1 Descripción del modelo

La base de datos está compuesta por nueve tablas. Todas cuentan con clave primaria numérica generada mediante secuencias, y las relaciones se implementan con claves foráneas. Los precios, costos y subtotales se almacenan en columnas NUMBER con dos decimales.

Además de las validaciones realizadas en Java, la base de datos protege las reglas del negocio mediante restricciones:

- **UNIQUE:** no se permiten placas ni DPI duplicados.
- **CHECK:** precios, costos y existencias no pueden ser negativos, y el estado de una orden solo puede tomar uno de los cuatro valores válidos.
- **Claves foráneas:** impiden eliminar un cliente con vehículos asociados, un vehículo con órdenes registradas o un mecánico con órdenes asignadas.

En los detalles de cada orden se guarda el precio vigente al momento de agregar el servicio o repuesto, de modo que un cambio posterior de precio no altera las órdenes ya registradas.

5.2 Diagrama entidad-relación

El diagrama muestra las nueve tablas con sus columnas, tipos de dato y restricciones (PK: llave primaria, FK: llave foránea, UQ: valor único, CK: restricción CHECK). Las líneas indican relaciones de uno a muchos: por ejemplo, un cliente tiene cero o muchos vehículos y una orden incluye cero o muchos servicios y repuestos.

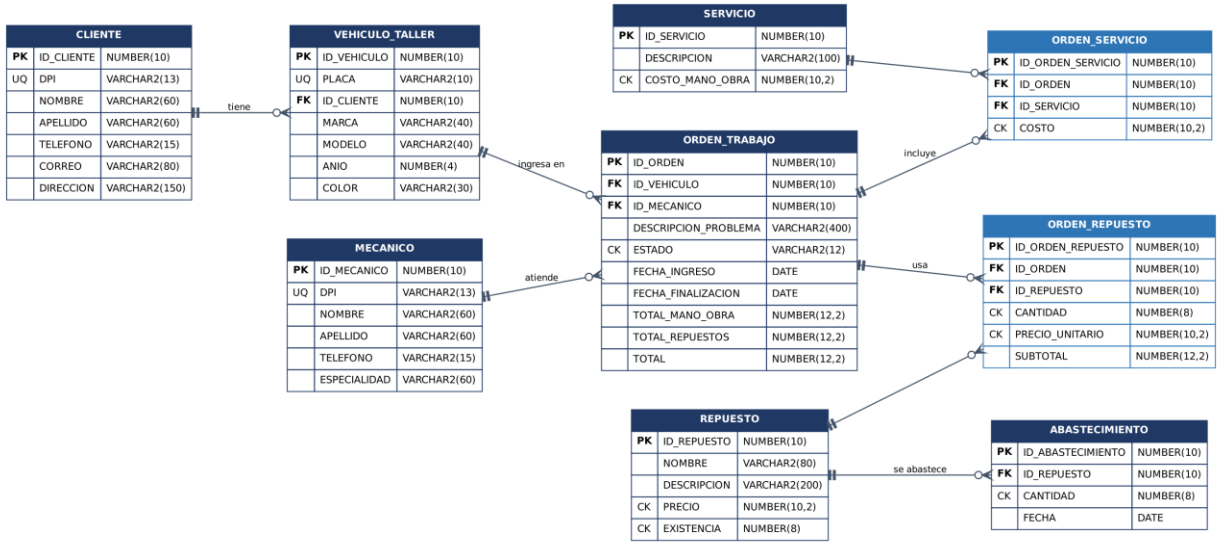


Figura 1. Diagrama entidad-relación de la base de datos del taller.

5.3 Diccionario de datos

CLIENTE

Clientes del taller.

Columna	Tipo	Restricciones
ID_CLIENTE	NUMBER(10)	PK, generado con seq_cliente
DPI	VARCHAR2(13)	NOT NULL, UNIQUE (identificador no duplicado)
NOMBRE	VARCHAR2(60)	NOT NULL
APELLIDO	VARCHAR2(60)	NOT NULL
TELEFONO	VARCHAR2(15)	
CORREO	VARCHAR2(80)	
DIRECCION	VARCHAR2(150)	

MECANICO

Mecánicos que atienden las órdenes.

Columna	Tipo	Restricciones
ID_MECANICO	NUMBER(10)	PK, generado con seq_mecanico
DPI	VARCHAR2(13)	NOT NULL, UNIQUE
NOMBRE	VARCHAR2(60)	NOT NULL
APELLIDO	VARCHAR2(60)	NOT NULL
TELEFONO	VARCHAR2(15)	
ESPECIALIDAD	VARCHAR2(60)	

VEHICULO_TALLER

Vehículos de los clientes.

Columna	Tipo	Restricciones
ID_VEHICULO	NUMBER(10)	PK, generado con seq_vehiculo
PLACA	VARCHAR2(10)	NOT NULL, UNIQUE (no se repiten placas)
ID_CLIENTE	NUMBER(10)	FK → CLIENTE
MARCA	VARCHAR2(40)	NOT NULL
MODELO	VARCHAR2(40)	NOT NULL
ANIO	NUMBER(4)	CHECK entre 1950 y 2100
COLOR	VARCHAR2(30)	

REPUESTO

Inventario de repuestos.

Columna	Tipo	Restricciones
ID_REPUESTO	NUMBER(10)	PK, generado con seq_repuesto
NOMBRE	VARCHAR2(80)	NOT NULL
DESCRIPCION	VARCHAR2(200)	
PRECIO	NUMBER(10,2)	CHECK PRECIO >= 0
EXISTENCIA	NUMBER(8)	DEFAULT 0, CHECK EXISTENCIA >= 0

ABASTECIMIENTO

Registro de cada ingreso de unidades al inventario.

Columna	Tipo	Restricciones
ID_ABASTECIMIENTO	NUMBER(10)	PK, generado con seq_abastecimiento
ID_REPUESTO	NUMBER(10)	FK → REPUESTO
CANTIDAD	NUMBER(8)	CHECK CANTIDAD > 0
FECHA	DATE	DEFAULT SYSDATE

SERVICIO

Servicios que ofrece el taller.

Columna	Tipo	Restricciones
ID_SERVICIO	NUMBER(10)	PK, generado con seq_servicio
DESCRIPCION	VARCHAR2(100)	NOT NULL
COSTO_MANO_OBRA	NUMBER(10,2)	CHECK >= 0

ORDEN_TRABAJO

Órdenes de trabajo (módulo principal).

Columna	Tipo	Restricciones
ID_ORDEN	NUMBER(10)	PK, generado con seq_orden
ID_VEHICULO	NUMBER(10)	FK → VEHICULO_TALLER
ID_MECANICO	NUMBER(10)	FK → MECANICO
DESCRIPCION_PROBLEMA	VARCHAR2(400)	NOT NULL
ESTADO	VARCHAR2(12)	DEFAULT PENDIENTE; CHECK PENDIENTE, EN PROCESO, FINALIZADA, CANCELADA
FECHA_INGRESO	DATE	DEFAULT SYSDATE
FECHA_FINALIZACION	DATE	Se llena al finalizar
TOTAL_MANO_OBRA	NUMBER(12,2)	Suma de servicios

Columna	Tipo	Restricciones
TOTAL_REPUESTOS	NUMBER(12,2)	Suma de subtotales de repuestos
TOTAL	NUMBER(12,2)	Mano de obra + repuestos

ORDEN_SERVICIO

Servicios incluidos en cada orden.

Columna	Tipo	Restricciones
ID_ORDEN_SERVICIO	NUMBER(10)	PK, generado con seq_orden_servicio
ID_ORDEN	NUMBER(10)	FK → ORDEN_TRABAJO
ID_SERVICIO	NUMBER(10)	FK → SERVICIO; UNIQUE con ID_ORDEN
COSTO	NUMBER(10,2)	Costo del servicio al momento de agregarlo

ORDEN_REPUESTO

Repuestos utilizados en cada orden.

Columna	Tipo	Restricciones
ID_ORDEN_REPUESTO	NUMBER(10)	PK, generado con seq_orden_repuesto
ID_ORDEN	NUMBER(10)	FK → ORDEN_TRABAJO
ID_REPUESTO	NUMBER(10)	FK → REPUESTO
CANTIDAD	NUMBER(8)	CHECK CANTIDAD > 0
PRECIO_UNITARIO	NUMBER(10,2)	Precio al momento de agregarlo
SUBTOTAL	NUMBER(12,2)	CANTIDAD × PRECIO_UNITARIO

5.4 Prueba de conexión

Antes de desarrollar los módulos se verificó la conexión de Java con Oracle mediante JDBC, consultando la cantidad de clientes registrados en los datos de prueba.

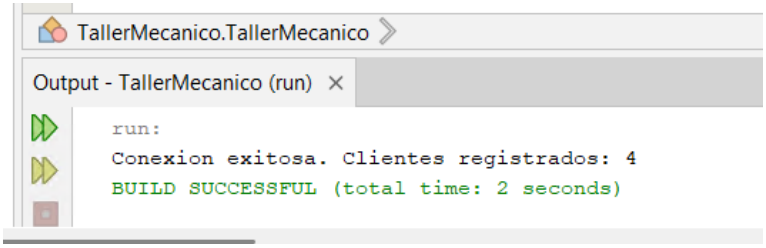


Figura 2. Prueba de conexión a Oracle desde NetBeans.

5.5 Script SQL de creación y datos de prueba

El siguiente script crea las secuencias y tablas del sistema e inserta datos de prueba: clientes, mecánicos, vehículos, repuestos (incluyendo repuestos con bajo inventario y uno agotado), servicios y órdenes en cada uno de los cuatro estados.

```
CREATE SEQUENCE seq_cliente START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_mecanico START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_vehiculo START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_repuesto START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_abastecimiento START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_servicio START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_orden START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_orden_servicio START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_orden_repuesto START WITH 1 INCREMENT BY 1 NOCACHE;


CREATE TABLE CLIENTE (
    ID_CLIENTE   NUMBER(10)     PRIMARY KEY,
    DPI          VARCHAR2(13)   NOT NULL,
    NOMBRE       VARCHAR2(60)   NOT NULL,
    APELLIDO     VARCHAR2(60)   NOT NULL,
    TELEFONO     VARCHAR2(15),
    CORREO       VARCHAR2(80),
    DIRECCION    VARCHAR2(150),
    CONSTRAINT UQ_CT_DPI UNIQUE (DPI)
);

CREATE TABLE MECANICO (
    ID_MECANICO  NUMBER(10)     PRIMARY KEY,
    DPI          VARCHAR2(13)   NOT NULL,
    NOMBRE       VARCHAR2(60)   NOT NULL,
    APELLIDO     VARCHAR2(60)   NOT NULL,
    TELEFONO     VARCHAR2(15),
    ESPECIALIDAD VARCHAR2(60),
    CONSTRAINT UQ_MECANICO_DPI UNIQUE (DPI)
);

CREATE TABLE VEHICULO_TALLER (
    ID_VEHICULO  NUMBER(10)     PRIMARY KEY,
    PLACA        VARCHAR2(10)   NOT NULL,
    ID_CLIENTE   NUMBER(10)     NOT NULL,
    MARCA        VARCHAR2(40)   NOT NULL,
    MODELO       VARCHAR2(40)   NOT NULL,
    ANIO         NUMBER(4)      NOT NULL,
    COLOR        VARCHAR2(30),
    CONSTRAINT UQ_VT_PLACA UNIQUE (PLACA),
    CONSTRAINT FK_VT_CLIENTE FOREIGN KEY (ID_CLIENTE) REFERENCES CLIENTE (ID_CLIENTE),
    CONSTRAINT CK_VT_ANIO CHECK (ANIO BETWEEN 1950 AND 2100)
);

CREATE TABLE REPUESTO (
    ID_REPUESTO  NUMBER(10)     PRIMARY KEY,
    NOMBRE       VARCHAR2(80)   NOT NULL,
    DESCRIPCION  VARCHAR2(200),
    PRECIO       NUMBER(10,2)   NOT NULL,
    EXISTENCIA   NUMBER(8)      DEFAULT 0 NOT NULL,
    CONSTRAINT CK_REPUESTO_PRECIO CHECK (PRECIO >= 0),
    CONSTRAINT CK_REPUESTO_EXISTENCIA CHECK (EXISTENCIA >= 0)
);

CREATE TABLE ABASTECIMIENTO (
    ID_ABASTECIMIENTO NUMBER(10) PRIMARY KEY,
    ID_REPUESTO  NUMBER(10)     NOT NULL,
    CANTIDAD     NUMBER(8)      NOT NULL,
    FECHA        DATE           DEFAULT SYSDATE NOT NULL,
    CONSTRAINT FK_ABAST_REPUESTO FOREIGN KEY (ID_REPUESTO) REFERENCES REPUESTO (ID_REPUESTO),
    CONSTRAINT CK_ABAST_CANTIDAD CHECK (CANTIDAD > 0)
);

CREATE TABLE SERVICIO (
```

```
    ID_SERVICIO     NUMBER(10)   PRIMARY KEY,
    DESCRIPCION     VARCHAR2(100) NOT NULL,
    COSTO_MANO_OBRA NUMBER(10,2) NOT NULL,
    CONSTRAINT CK_SERVICIO_COSTO CHECK (COSTO_MANO_OBRA >= 0)
);

CREATE TABLE ORDEN_TRABAJO (
    ID_ORDEN             NUMBER(10)    PRIMARY KEY,
    ID_VEHICULO          NUMBER(10)    NOT NULL,
    ID_MECANICO          NUMBER(10)    NOT NULL,
    DESCRIPCION_PROBLEMA VARCHAR2(400) NOT NULL,
    ESTADO               VARCHAR2(12)  DEFAULT 'PENDIENTE' NOT NULL,
    FECHA_INGRESO        DATE          DEFAULT SYSDATE NOT NULL,
    FECHA_FINALIZACION   DATE,
    TOTAL_MANO_OBRA      NUMBER(12,2)  DEFAULT 0 NOT NULL,
    TOTAL_REPUESTOS      NUMBER(12,2)  DEFAULT 0 NOT NULL,
    TOTAL                NUMBER(12,2)  DEFAULT 0 NOT NULL,
    CONSTRAINT FK_OT_VEHICULO FOREIGN KEY (ID_VEHICULO) REFERENCES VEHICULO_TALLER (ID_VEHICULO),
    CONSTRAINT FK_OT_MECANICO FOREIGN KEY (ID_MECANICO) REFERENCES MECANICO (ID_MECANICO),
    CONSTRAINT CK_OT_ESTADO CHECK (ESTADO IN ('PENDIENTE', 'EN PROCESO', 'FINALIZADA', 'CANCELADA'))
);

CREATE TABLE ORDEN_SERVICIO (
    ID_ORDEN_SERVICIO NUMBER(10)   PRIMARY KEY,
    ID_ORDEN          NUMBER(10)   NOT NULL,
    ID_SERVICIO       NUMBER(10)   NOT NULL,
    COSTO             NUMBER(10,2) NOT NULL,
    CONSTRAINT FK_OS_ORDEN FOREIGN KEY (ID_ORDEN) REFERENCES ORDEN_TRABAJO (ID_ORDEN),
    CONSTRAINT FK_OS_SERVICIO FOREIGN KEY (ID_SERVICIO) REFERENCES SERVICIO (ID_SERVICIO),
    CONSTRAINT UQ_OS UNIQUE (ID_ORDEN, ID_SERVICIO),
    CONSTRAINT CK_OS_COSTO CHECK (COSTO >= 0)
);

CREATE TABLE ORDEN_REPUESTO (
    ID_ORDEN_REPUESTO NUMBER(10)   PRIMARY KEY,
    ID_ORDEN          NUMBER(10)   NOT NULL,
    ID_REPUESTO       NUMBER(10)   NOT NULL,
    CANTIDAD          NUMBER(8)    NOT NULL,
    PRECIO_UNITARIO   NUMBER(10,2) NOT NULL,
    SUBTOTAL          NUMBER(12,2) NOT NULL,
    CONSTRAINT FK_OR_ORDEN FOREIGN KEY (ID_ORDEN) REFERENCES ORDEN_TRABAJO (ID_ORDEN),
    CONSTRAINT FK_OR_REPUESTO FOREIGN KEY (ID_REPUESTO) REFERENCES REPUESTO (ID_REPUESTO),
    CONSTRAINT CK_OR_CANTIDAD CHECK (CANTIDAD > 0),
    CONSTRAINT CK_OR_PRECIO CHECK (PRECIO_UNITARIO >= 0)
);

INSERT INTO CLIENTE VALUES (seq_cliente.NEXTVAL, '2547896320101', 'Carlos', 'López', '55123456',
'carlos.lopez@gmail.com', 'Zona 1, Villa Nueva');
INSERT INTO CLIENTE VALUES (seq_cliente.NEXTVAL, '3012457890101', 'María', 'González', '44987654',
'maria.gonzalez@gmail.com', 'Zona 5, Mixco');
INSERT INTO CLIENTE VALUES (seq_cliente.NEXTVAL, '1987456320101', 'José', 'Ramírez', '33456789',
'jose.ramirez@gmail.com', 'Zona 12, Guatemala');
INSERT INTO CLIENTE VALUES (seq_cliente.NEXTVAL, '2765432100101', 'Ana', 'Pérez', '52345678', NULL, 'Zona 4,
Villa Nueva');

INSERT INTO MECANICO VALUES (seq_mecanico.NEXTVAL, '2456789010101', 'Luis', 'Méndez', '51234567', 'Motor');
INSERT INTO MECANICO VALUES (seq_mecanico.NEXTVAL, '2987654320101', 'Pedro', 'Castillo', '42345678', 'Frenos y
suspensión');
INSERT INTO MECANICO VALUES (seq_mecanico.NEXTVAL, '3124567890101', 'Jorge', 'Hernández', '30987654',
'Electricidad');

INSERT INTO VEHICULO_TALLER VALUES (seq_vehiculo.NEXTVAL, 'P123ABC', 1, 'Toyota', 'Corolla', 2018, 'Blanco');
INSERT INTO VEHICULO_TALLER VALUES (seq_vehiculo.NEXTVAL, 'P456DEF', 1, 'Honda', 'Civic', 2020, 'Negro');
INSERT INTO VEHICULO_TALLER VALUES (seq_vehiculo.NEXTVAL, 'P789GHI', 2, 'Mazda', 'CX-5', 2019, 'Rojo');
INSERT INTO VEHICULO_TALLER VALUES (seq_vehiculo.NEXTVAL, 'M321JKL', 3, 'Nissan', 'Frontier', 2016, 'Gris');

INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Filtro de aceite', 'Filtro de aceite universal', 45.00, 20);
INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Aceite 15W40', 'Galón de aceite para motor', 120.00, 15);
INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Pastillas de freno', 'Juego de pastillas delanteras', 250.00,
4);
```

```
INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Bujía', 'Bujía de iridio', 35.00, 30);
INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Filtro de aire', 'Filtro de aire de motor', 80.00, 0);
INSERT INTO REPUESTO VALUES (seq_repuesto.NEXTVAL, 'Batería 12V', 'Batería de 12 voltios 60Ah', 650.00, 3);


INSERT INTO ABASTECIMIENTO VALUES (seq_abastecimiento.NEXTVAL, 1, 21, DATE '2026-09-28');
INSERT INTO ABASTECIMIENTO VALUES (seq_abastecimiento.NEXTVAL, 2, 16, DATE '2026-09-28');
INSERT INTO ABASTECIMIENTO VALUES (seq_abastecimiento.NEXTVAL, 3, 5, DATE '2026-09-28');
INSERT INTO ABASTECIMIENTO VALUES (seq_abastecimiento.NEXTVAL, 4, 30, DATE '2026-09-28');
INSERT INTO ABASTECIMIENTO VALUES (seq_abastecimiento.NEXTVAL, 6, 3, DATE '2026-09-28');


INSERT INTO SERVICIO VALUES (seq_servicio.NEXTVAL, 'Cambio de aceite', 100.00);
INSERT INTO SERVICIO VALUES (seq_servicio.NEXTVAL, 'Revisión de frenos', 150.00);
INSERT INTO SERVICIO VALUES (seq_servicio.NEXTVAL, 'Alineación', 125.00);
INSERT INTO SERVICIO VALUES (seq_servicio.NEXTVAL, 'Reparación de motor', 1500.00);
INSERT INTO SERVICIO VALUES (seq_servicio.NEXTVAL, 'Diagnóstico eléctrico', 200.00);


INSERT INTO ORDEN_TRABAJO VALUES (seq_orden.NEXTVAL, 1, 1, 'Cambio de aceite y ruido al frenar', 'FINALIZADA',
DATE '2026-10-01', DATE '2026-10-02', 250.00, 165.00, 415.00);
INSERT INTO ORDEN_TRABAJO VALUES (seq_orden.NEXTVAL, 3, 2, 'El vehículo se jala hacia la derecha', 'EN PROCESO',
DATE '2026-10-08', NULL, 125.00, 250.00, 375.00);
INSERT INTO ORDEN_TRABAJO VALUES (seq_orden.NEXTVAL, 4, 3, 'No enciende por las mañanas', 'PENDIENTE', DATE
'2026-10-12', NULL, 0, 0, 0);
INSERT INTO ORDEN_TRABAJO VALUES (seq_orden.NEXTVAL, 2, 1, 'Revisión general', 'CANCELADA', DATE '2026-10-05',
NULL, 0, 0, 0);


INSERT INTO ORDEN_SERVICIO VALUES (seq_orden_servicio.NEXTVAL, 1, 1, 100.00);
INSERT INTO ORDEN_SERVICIO VALUES (seq_orden_servicio.NEXTVAL, 1, 2, 150.00);
INSERT INTO ORDEN_SERVICIO VALUES (seq_orden_servicio.NEXTVAL, 2, 3, 125.00);


INSERT INTO ORDEN_REPUESTO VALUES (seq_orden_repuesto.NEXTVAL, 1, 1, 1, 45.00, 45.00);
INSERT INTO ORDEN_REPUESTO VALUES (seq_orden_repuesto.NEXTVAL, 1, 2, 1, 120.00, 120.00);
INSERT INTO ORDEN_REPUESTO VALUES (seq_orden_repuesto.NEXTVAL, 2, 3, 1, 250.00, 250.00);


COMMIT;
```

6. Diagrama de clases UML

6.1 Clases del modelo

El diagrama muestra las clases del modelo con sus atributos privados y métodos públicos. Las flechas con punta vacía representan herencia (Cliente y Mecanico heredan de Persona; DetalleServicio y DetalleRepuesto de Detalle; InventarioInsuficienteException de TallerException). El rombo relleno representa la composición: una OrdenTrabajo contiene sus detalles de servicios y repuestos. Las líneas punteadas indican las referencias por identificador entre clases.

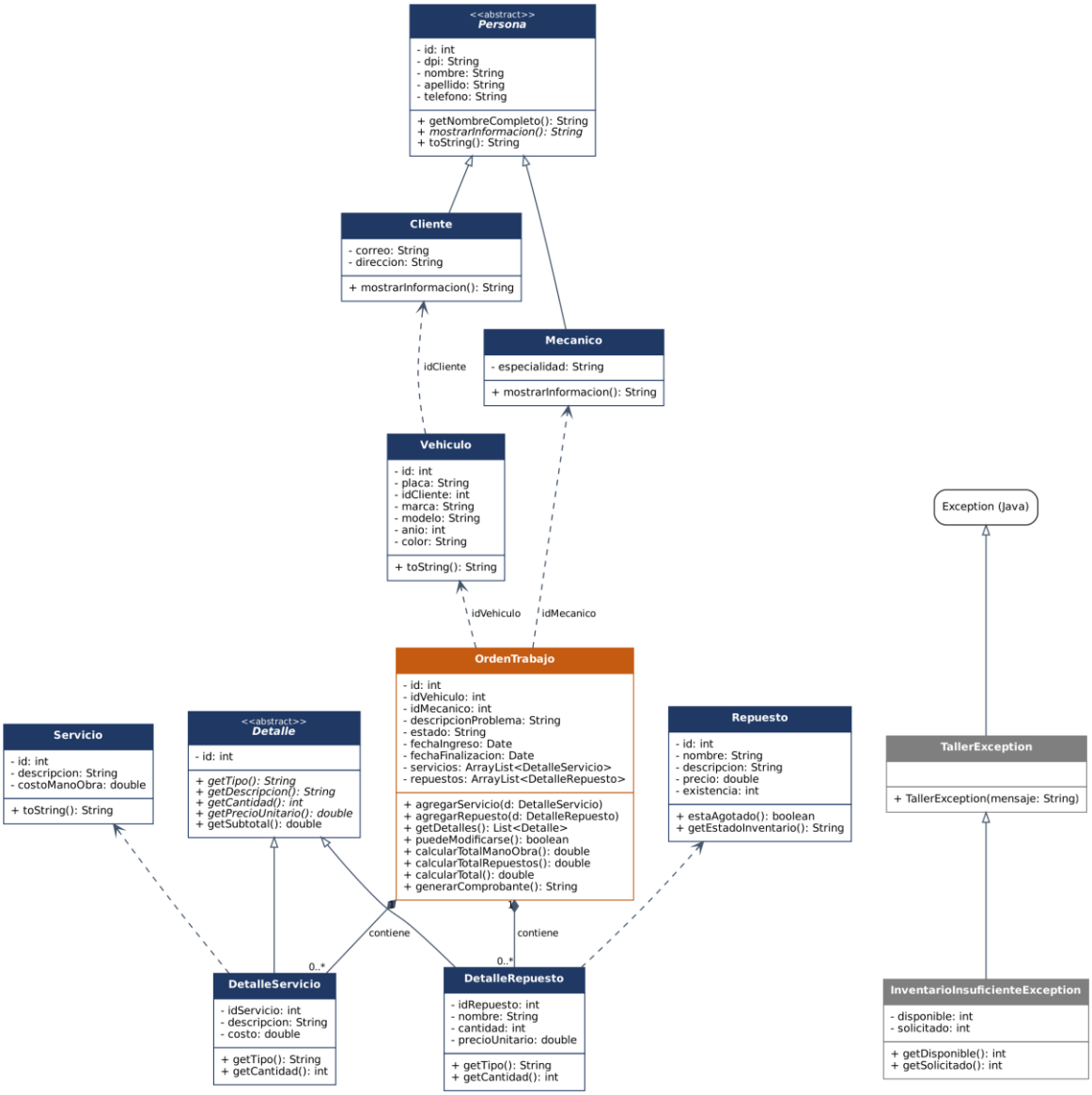


Figura 3. Diagrama de clases del modelo.

6.2 Arquitectura del sistema

El sistema está organizado en capas. Los formularios (capa de presentación) solo manejan la interfaz y llaman a los gestores; los gestores (capa de acceso a datos) contienen todas las sentencias SQL con PreparedStatement y obtienen la conexión mediante la clase Conexion, que se comunica con Oracle por JDBC.

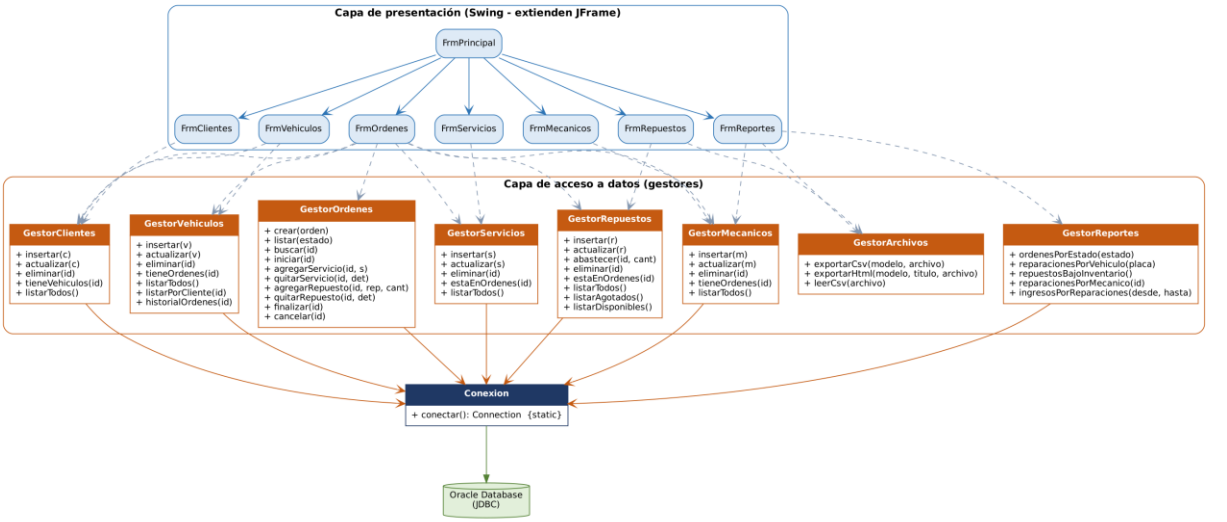


Figura 4. Diagrama de arquitectura: formularios, gestores y conexión.

7. Aplicación de la programación orientada a objetos

Concepto	Aplicación en el proyecto
Encapsulamiento	Todas las clases del modelo (Persona, Cliente, Vehiculo, Repuesto, OrdenTrabajo, etc.) tienen atributos privados con métodos get y set.
Herencia	Persona es una clase abstracta de la que heredan Cliente y Mecanico. Detalle es abstracta y la heredan DetalleServicio y DetalleRepuesto. InventarioInsuficienteException hereda de TallerException. Los formularios heredan de JFrame.
Polimorfismo	Cliente y Mecanico sobrescriben mostrarInformacion(). DetalleServicio y DetalleRepuesto sobrescriben getTipo(), getDescripcion(), getCantidad() y getPrecioUnitario(); el comprobante recorre una List<Detalle> y calcula cada subtotal sin conocer el tipo concreto.
ArrayList	Los gestores devuelven listas de objetos (List<Cliente>, List<Vehiculo>, etc.) y OrdenTrabajo administra sus detalles en dos ArrayList.
Composición	OrdenTrabajo contiene sus listas de DetalleServicio y DetalleRepuesto, y calcula a partir de ellas los totales de mano de obra, repuestos y el total general.
Manejo de excepciones	Los gestores lanzan SQLException y las excepciones propias TallerException e InventarioInsuficienteException; los formularios las atrapan con try/catch y muestran mensajes. También se controla NumberFormatException en los datos de entrada.

El acceso a datos está separado en clases gestoras (GestorClientes, GestorVehiculos, GestorOrdenes, etc.), de modo que los formularios no contienen sentencias SQL. Todas las operaciones usan PreparedStatement.

8. Módulos del sistema

La aplicación inicia en un menú principal desde el cual se abre cada módulo en su propia ventana. Al cerrar un módulo solo se cierra su ventana; al cerrar el menú termina la aplicación.



Figura 5. Menú principal.

8.1 Clientes

Permite registrar, modificar, consultar y eliminar clientes. Valida que el DPI tenga 13 dígitos y que no se repita (restricción UNIQUE). No permite eliminar un cliente que tenga vehículos registrados y muestra un mensaje con el motivo.

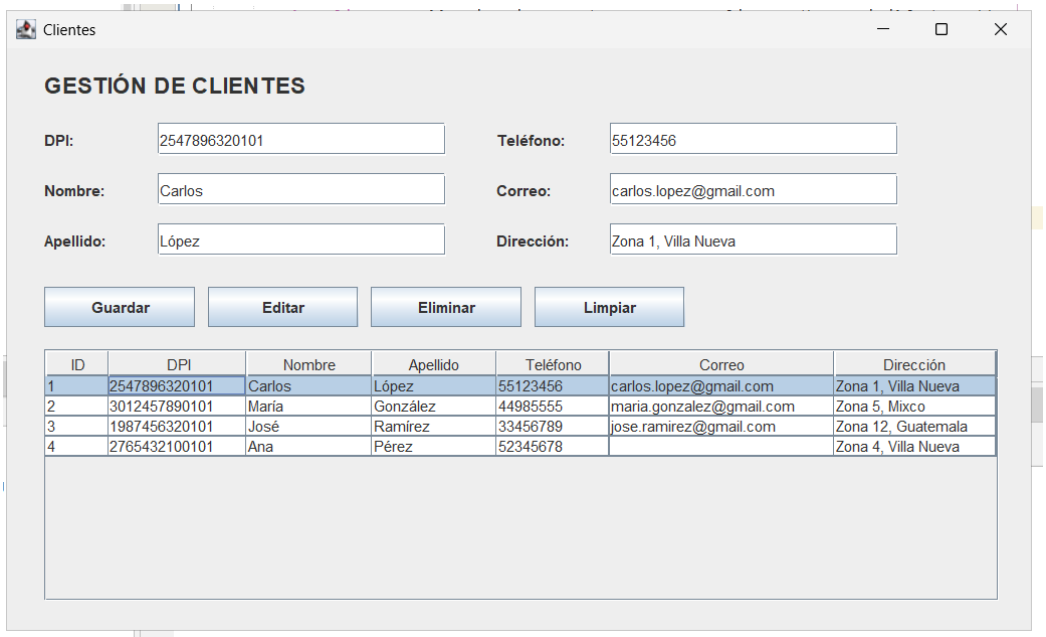


Figura 6. Gestión de clientes.

8.2 Vehículos

Registra vehículos asociados a un cliente existente, que se selecciona en una lista desplegable. Valida el formato de la placa y que no se repita. Permite filtrar los vehículos de un cliente y consultar el historial de órdenes de un vehículo. No permite eliminar vehículos con órdenes registradas.

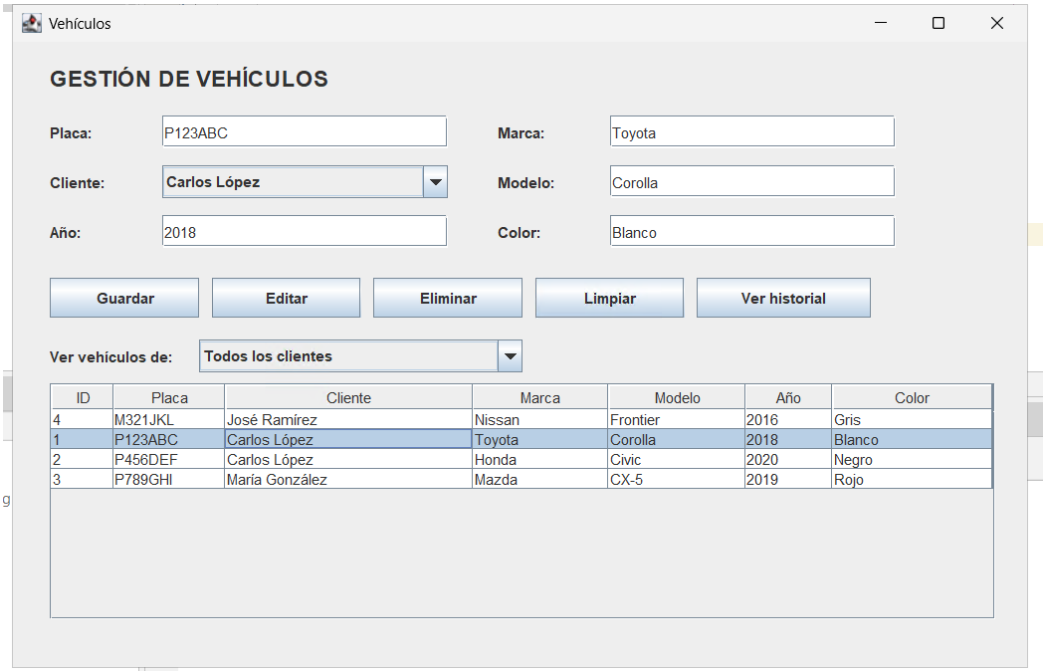


Figura 7. Gestión de vehículos.

8.3 Mecánicos

Permite registrar, modificar, consultar y eliminar mecánicos con su especialidad. Un mecánico puede atender varias órdenes al mismo tiempo, y no se permite eliminar un mecánico con órdenes asociadas.

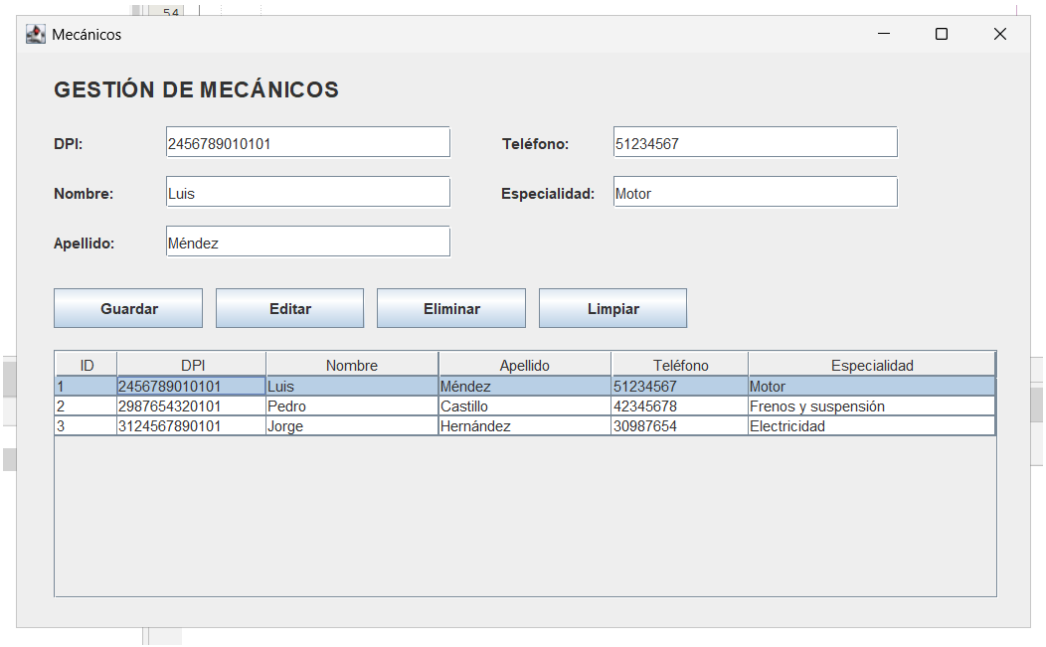


Figura 8. Gestión de mecánicos.

8.4 Repuestos

Registra los repuestos con nombre, descripción, precio y existencia inicial. Permite modificar su información, abastecer el inventario (suma unidades y registra la cantidad ingresada en ABASTECIMIENTO), consultar existencias e identificar los agotados con el botón Ver agotados. La columna Estado indica AGOTADO, BAJO (cinco o menos) o DISPONIBLE. No se permiten precios ni cantidades negativas. También permite importar repuestos desde un archivo CSV.



Figura 9. Gestión de repuestos.

8.5 Servicios

Registra los servicios que ofrece el taller con su descripción y costo de mano de obra. No permite costos negativos ni eliminar servicios que ya se utilizaron en órdenes.

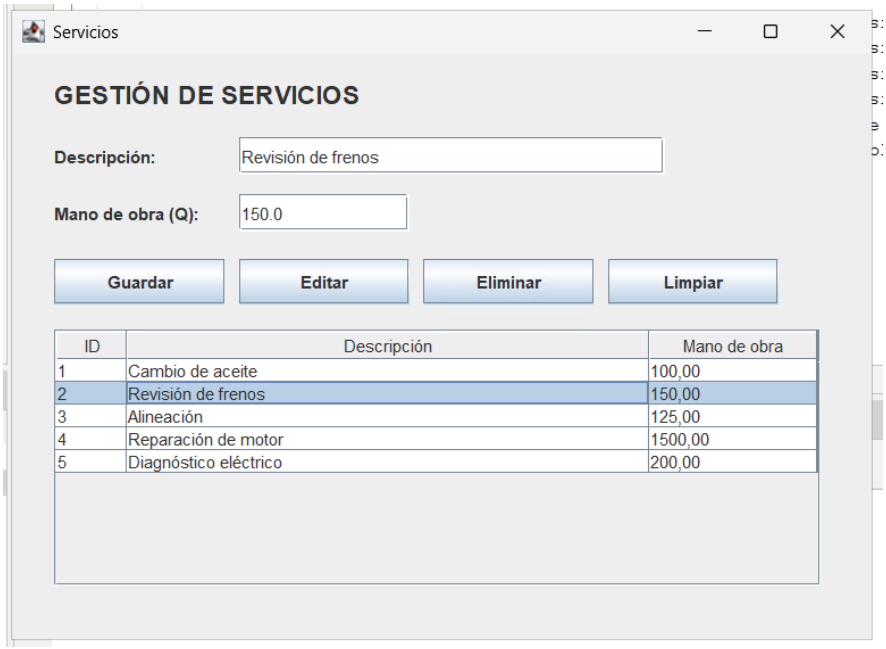


Figura 10. Gestión de servicios.

8.6 Órdenes de trabajo

Es el módulo principal. En la parte superior izquierda se crea la orden eligiendo el cliente, uno de sus vehículos y el mecánico, e ingresando la descripción del problema. A la derecha se listan las órdenes con filtro por estado. Al seleccionar una orden se muestran sus servicios y repuestos, los totales de mano de obra y repuestos y el total general. Los botones se activan según el estado: en PENDIENTE solo se puede iniciar o cancelar; en EN PROCESO se agregan y quitan servicios y repuestos, se finaliza o se cancela; una orden FINALIZADA solo permite ver el comprobante y una CANCELADA no admite cambios.

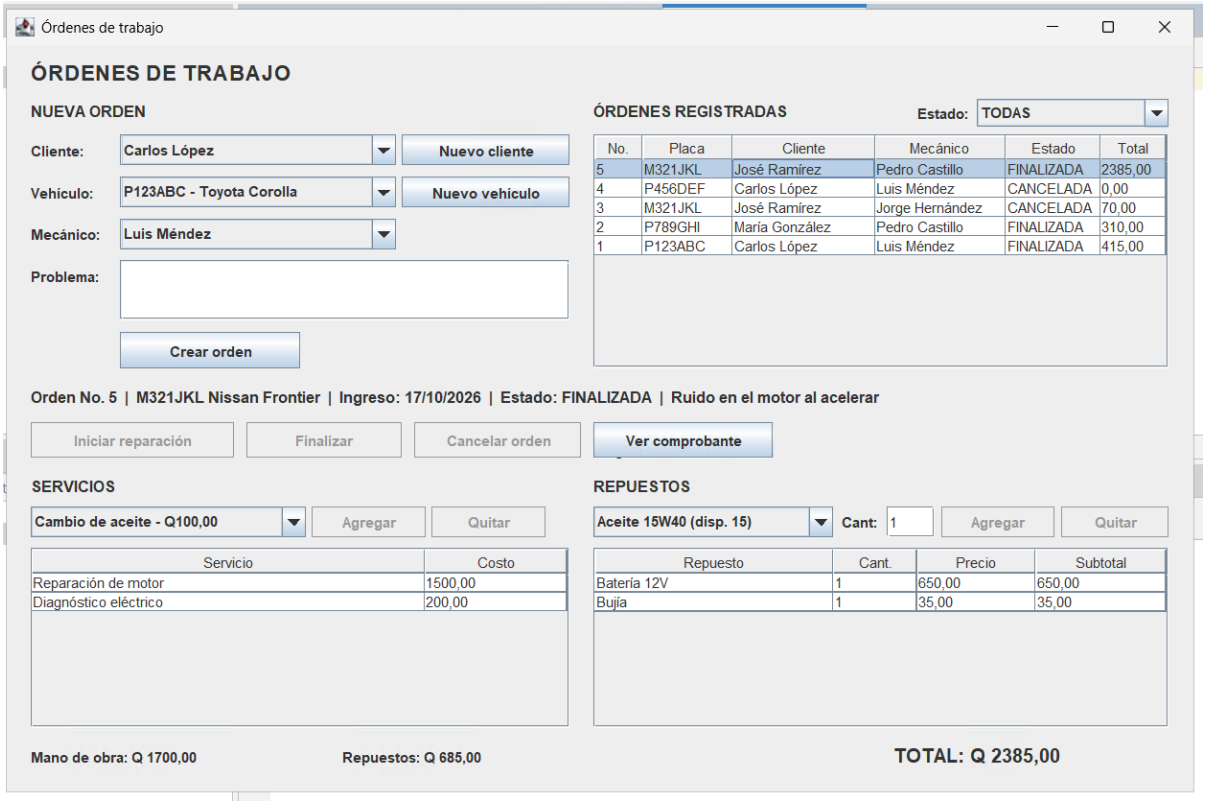


Figura 11. Módulo de órdenes de trabajo con una orden finalizada.

8.7 Transacciones

Las operaciones que modifican varias tablas se ejecutan como una única transacción: se desactiva el auto-commit con setAutoCommit(false), se ejecutan todas las sentencias y se confirma con commit(); si ocurre cualquier error se ejecuta rollback() y no se aplica ningún cambio.

- **Asignación de repuestos:** se bloquea la orden y el repuesto con SELECT ... FOR UPDATE, se verifica que la orden esté EN PROCESO y que la cantidad sea mayor que cero y no supere la existencia. Si no hay inventario suficiente se lanza InventarioInsuficienteException y se hace rollback. Si hay, se descuenta la existencia (UPDATE ... WHERE EXISTENCIA >= cantidad), se inserta el detalle en ORDEN_REPUESTO y se recalculan los totales, todo en la misma transacción.
- **Cancelación de órdenes:** se verifica que la orden esté PENDIENTE o EN PROCESO, se devuelven al inventario las cantidades de todos los repuestos asignados y se cambia el estado a CANCELADA, dentro de una sola transacción.
- **Quitar un repuesto:** al quitar un repuesto de una orden en proceso, se devuelven sus unidades al inventario, se elimina el detalle y se recalculan los totales en una transacción.
- **Abastecimiento:** el abastecimiento suma unidades a la existencia y registra la cantidad en ABASTECIMIENTO en una sola transacción.

En la siguiente figura se solicitaron 3 baterías cuando solo había 2 disponibles. El sistema rechazó la operación con InventarioInsuficienteException, se hizo rollback y la existencia se mantuvo sin cambios.

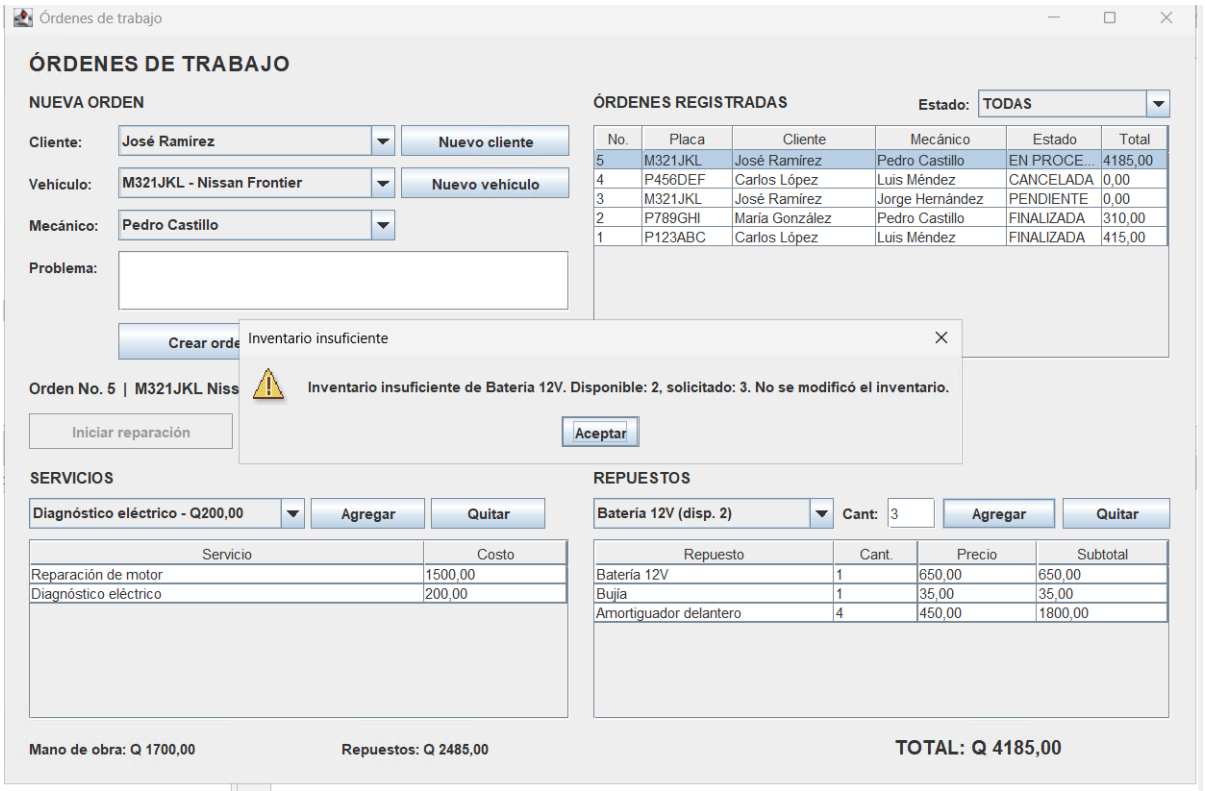


Figura 12. Asignación rechazada por inventario insuficiente.

9. Casos de uso

Los cuatro casos de uso obligatorios se probaron con los datos de prueba del script SQL.

Caso de uso	Prueba realizada	Resultado
1. Recepción de vehículo	Se seleccionó al cliente José Ramírez; el sistema mostró solo su vehículo M321JKL. Se asignó un mecánico, se ingresó el problema y se creó la orden. Los botones Nuevo cliente y Nuevo vehículo permiten registrarlos si no existen (flujo alterno).	Orden No. 5 creada en estado PENDIENTE.
2. Reparación del vehículo	Se inició la orden (EN PROCESO), se agregaron servicios y repuestos y se intentó agregar más unidades de las disponibles.	El inventario se descontó al agregar repuestos y los totales se actualizaron. La asignación con existencia insuficiente fue rechazada sin modificar el inventario (Figura 12).
3. Finalización	Se intentó finalizar una orden sin servicios y luego una con servicios.	La primera fue rechazada. La segunda pasó a FINALIZADA, se registró la fecha y se generó el comprobante con el detalle y el total a pagar.
4. Cancelación	Se canceló una orden en proceso que tenía repuestos asignados.	El sistema pidió confirmación, reintegró las unidades al inventario y cambió la orden a CANCELADA en una sola transacción.

10. Reportes

El módulo de reportes permite elegir uno de los cinco reportes obligatorios, indicar su filtro y mostrar el resultado en un JTable. Cada reporte puede exportarse a CSV (para abrirse en Excel) o a HTML.

Reporte	Filtro	Contenido
01 Órdenes por estado	Estado o todas	Número, placa, cliente, mecánico, fecha de ingreso, estado y total
02 Reparaciones por vehículo	Placa	Historial de órdenes del vehículo con fechas, mecánico, problema, estado y total
03 Repuestos con bajo inventario	Ninguno	Repuestos con existencia menor o igual a cinco, indicando si están agotados
04 Reparaciones por mecánico	Mecánico o todos	Órdenes asignadas a cada mecánico y su estado
05 Ingresos por reparaciones	Rango de fechas	Órdenes finalizadas en el rango con mano de obra, repuestos y total, más la suma general

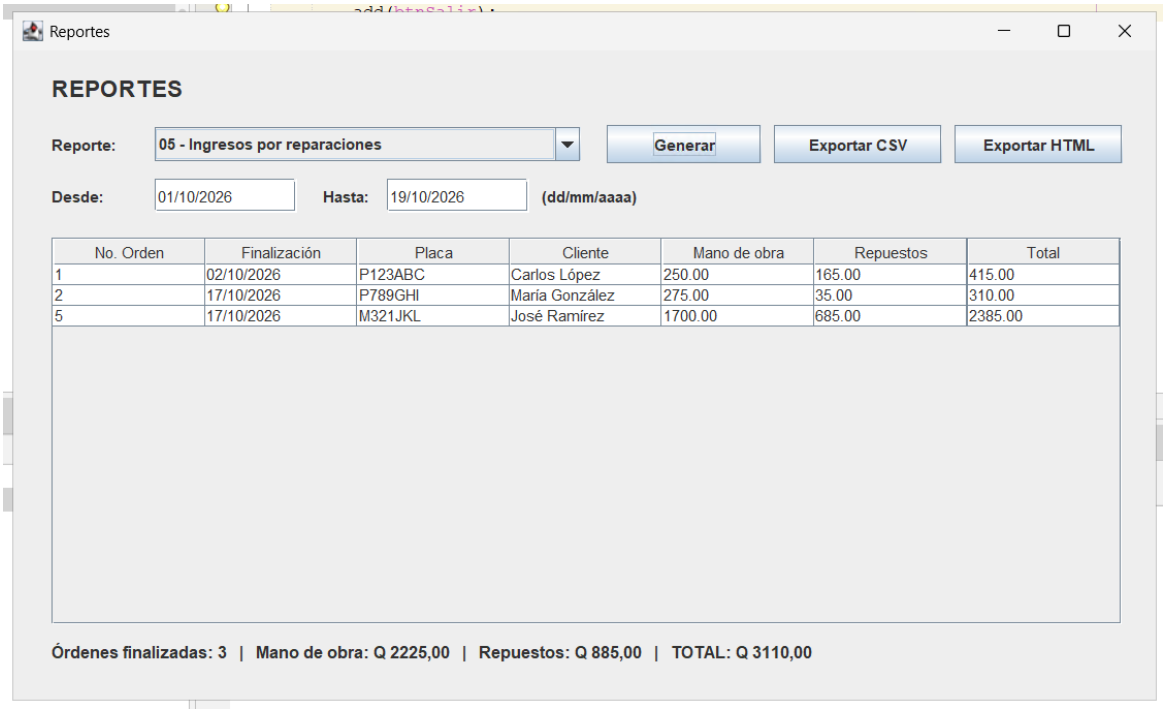


Figura 13. Reporte 05: ingresos por reparaciones del 01/10/2026 al 19/10/2026.

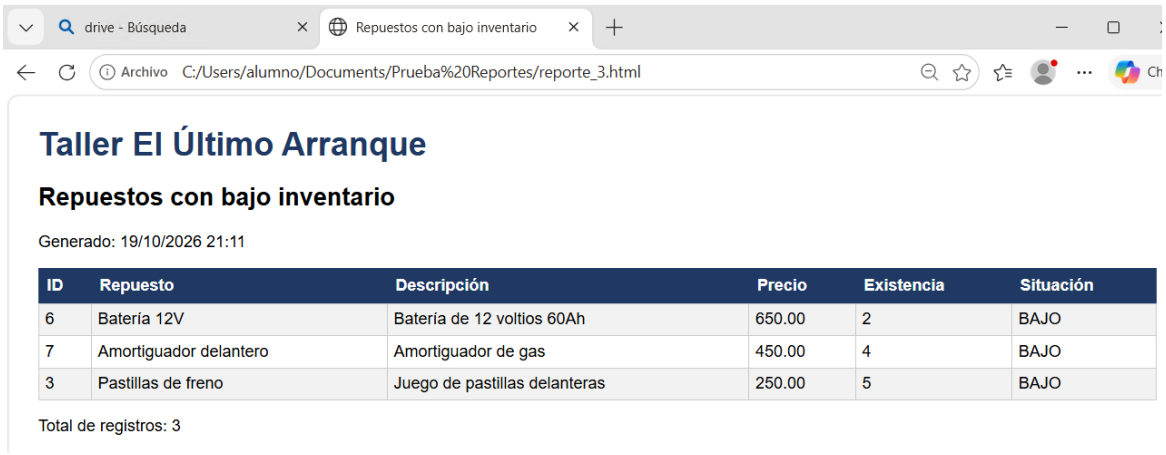


Figura 14. Reporte 03 exportado a HTML.

10.1 Importación y exportación CSV

La clase GestorArchivos se encarga de la lectura y escritura de archivos:

- **Exportación CSV:** cualquier reporte se guarda como archivo CSV en UTF-8, con encabezados y valores separados por coma. Los valores que contienen comas o comillas se encierran entre comillas. Al inicio del archivo se escribe la marca BOM para que Excel muestre correctamente las tildes.
- **Exportación HTML:** el reporte se guarda como una página HTML con una tabla, el título del reporte y la fecha de generación.
- **Importación CSV:** en el módulo de repuestos se puede cargar un archivo CSV con las columnas nombre, descripción, precio y existencia. El sistema omite el encabezado, valida cada línea y registra los repuestos válidos; al final informa cuántos se importaron y qué líneas tuvieron error.

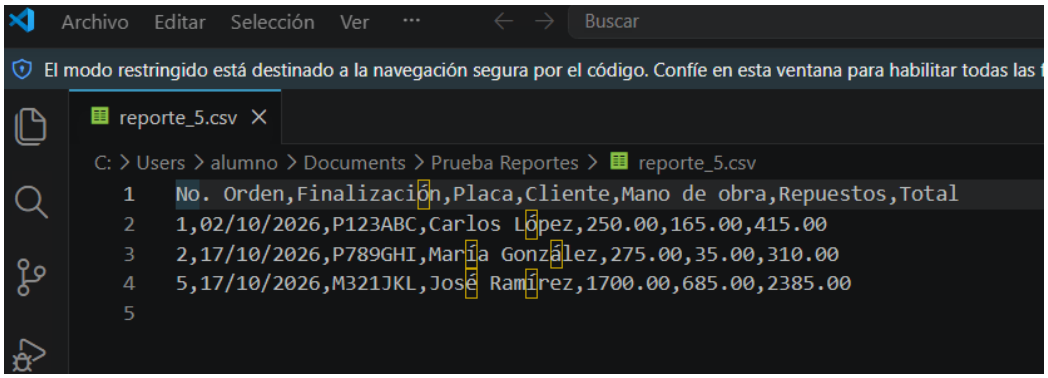


Figura 15. Reporte 05 exportado a CSV.

En la carpeta reportes del repositorio se incluyen reportes exportados en ambos formatos.

11. Repositorio Git

El código fuente del proyecto se administra con Git y se encuentra publicado en GitHub en la siguiente dirección:

https://github.com/omenendezm/ProyectoTallerMecanico

El repositorio contiene el proyecto de NetBeans, la carpeta src con el código Java y la carpeta sql con el script de creación de la base de datos (crear_tablas.sql), la carpeta diagramas con los diagramas ER y UML, y la carpeta reportes con ejemplos de reportes exportados. Mediante el archivo .gitignore se excluyen las carpetas generadas automáticamente al compilar (build y dist) y la configuración privada de NetBeans.

11.1 Historial de commits

No.	Fecha	Autor	Descripción
1	13/10/2026	omenendezm	Estructura inicial, conexión a Oracle y script SQL
2	14/10/2026	omenendezm	Persona, Cliente y Mecánico con gestores, formularios y menú principal
3	15/10/2026	omenendezm	Vehículos, repuestos con abastecimiento, servicios e importación CSV
4	17/10/2026	omenendezm	Órdenes de trabajo con estados, transacciones de repuestos y comprobante
5	19/10/2026	omenendezm	Módulo de reportes con exportación CSV y HTML, diagramas UML y ER

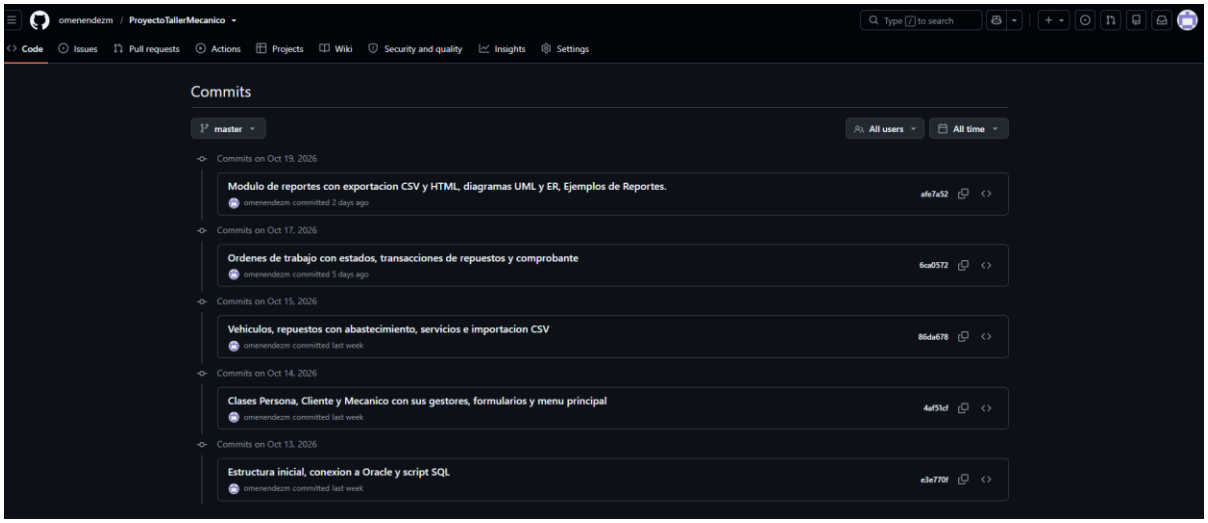


Figura 16. Historial de commits del repositorio en GitHub.

12. Conclusiones

- Se desarrolló una aplicación de escritorio en Java con Swing y Oracle Database que administra clientes, vehículos, mecánicos, repuestos, servicios y órdenes de trabajo, cumpliendo los requerimientos funcionales del enunciado.
- El uso de transacciones con commit y rollback garantiza que la asignación de repuestos y el descuento del inventario, así como la cancelación de órdenes y el reintegro de repuestos, se apliquen completos o no se apliquen, manteniendo la consistencia de los datos.
- La programación orientada a objetos permitió organizar el sistema: la herencia evita repetir código en Persona y Detalle, el polimorfismo permite tratar servicios y repuestos de forma uniforme en el comprobante y la composición refleja que una orden contiene sus detalles.
- Validar las reglas del negocio tanto en Java como en la base de datos (UNIQUE, CHECK y llaves foráneas) brinda una doble protección contra datos incorrectos.
- Separar el sistema en formularios, gestores y modelo facilita el mantenimiento: cada clase tiene una responsabilidad y los cambios en la base de datos se concentran en los gestores.
- El uso de Git con commits en distintos días permitió llevar un control del avance del proyecto y conservar su historial.